TP de Révision – MySQL Complet (Simulation Contrôle)

Correction :

Partie 1 : Création et manipulation de base et tables

Exercice 1 : Bases et tables

1. Créer une base tp_eshop_db **si elle n'existe pas**.

   CREATE DATABASE IF NOT EXISTS tp_eshop_db;

   USE tp_eshop_db;

2. Créer la table clients avec les colonnes suivantes :
   - id INT AUTO_INCREMENT, PRIMARY KEY
   - nom CHAR(50) NOT NULL
   - prenom CHAR(40)
   - date_naissance DATE
   - adresse VARCHAR(50) DEFAULT ''

```
CREATE TABLE clients (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom CHAR(50) NOT NULL,
  prenom CHAR(40),
  date_naissance DATE,
  adresse VARCHAR(50) DEFAULT ''
);
```

3. Ajouter une colonne email à la table clients.

   ALTER TABLE clients ADD email VARCHAR(100);

4. Supprimer une colonne inutile nommée pays (si elle existe).

ALTER TABLE clients DROP COLUMN pays;

5. Renommer la table clients en acheteurs.

   ALTER TABLE clients RENAME TO acheteurs;

Vérifier les modifications avec DESCRIBE.

DESCRIBE acheteurs;

Exercice 2 : Insertion et modification de données

1. Ajouter les clients suivants dans la table acheteurs :

nom	Prenom	date_naissance	adresse
Alami	Said	1980-05-22	Casablanca
Zaki	Fatima	1986-05-22	Agadir
Fadani	Ali	1990-08-10	Casablanca

```
INSERT INTO acheteurs (nom, prenom, date_naissance, adresse)
VALUES
('Alami','Said','1980-05-22','Casablanca'),
('Zaki','Fatima','1986-05-22','Agadir'),
('Fadani','Ali','1990-08-10','Casablanca');
```

2. Modifier le client avec id=1 pour changer son nom en Fadani et son adresse en Rabat.

```
UPDATE acheteurs
SET nom='Fadani', adresse='Rabat'
WHERE id=1;
```

3. Supprimer le client avec id=3.

   DELETE FROM acheteurs WHERE id=3;

4. Sélectionner tous les clients de Casablanca triés par nom.

   SELECT * FROM acheteurs

   WHERE adresse='Casablanca'

   ORDER BY nom;

Partie 2 : Vues et tables temporaires

1. Créer une **vue** vw_clients_agadir contenant tous les clients dont l'adresse est Agadir (insensible à la casse).

   CREATE VIEW vw_clients_agadir AS

   SELECT * FROM acheteurs

   WHERE LOWER(adresse) = 'agadir';

2. Créer une **table temporaire** temp_clients avec la structure de acheteurs mais sans données.

   CREATE TEMPORARY TABLE temp_clients LIKE acheteurs;

3. Supprimer la table temporaire.

   DROP TEMPORARY TABLE temp_clients;

Partie 3 :– Procédures et Fonctions

Exercice 3 : Procédures stockées

1. Créer une procédure sp_client_par_ville avec un paramètre d'entrée ville qui retourne tous les clients de cette ville (insensible à la casse).

```
DELIMITER $$
CREATE PROCEDURE sp_client_par_ville(IN ville VARCHAR(50))
BEGIN
  SELECT * FROM acheteurs
  WHERE LOWER(adresse) = LOWER(ville);
END$$
DELIMITER ;
```

2. Appeler la procédure pour la ville 'Agadir'.

```
CALL sp_client_par_ville('Agadir');
```

Exercice 4 : Fonctions

1. Créer une fonction uf_nombre_clients_par_ville(ville VARCHAR(50)) qui retourne le nombre de clients dans une ville.

```
DELIMITER $$
CREATE FUNCTION uf_nombre_clients_par_ville(ville VARCHAR(50))
RETURNS INT
DETERMINISTIC
READS SQL DATA
BEGIN
  DECLARE total INT;
  SELECT COUNT(*) INTO total
  FROM acheteurs
  WHERE LOWER(adresse)=LOWER(ville);
  RETURN total;
END$$
```

```
DELIMITER ;
```

2. Tester la fonction pour la ville 'Casablanca'.

   ```
   SELECT uf_nombre_clients_par_ville('Casablanca');
   ```

Partie 4 : Triggers

1. Ajouter une colonne date_modification à la table acheteurs.

   ```
   ALTER TABLE acheteurs ADD date_modification DATETIME;
   ```

2. Créer un trigger maj_date_modif qui met automatiquement date_modification à la date et heure actuelle **avant chaque mise à jour** sur acheteurs.

   ```
   DELIMITER $$
   CREATE TRIGGER maj_date_modif
   BEFORE UPDATE ON acheteurs
   FOR EACH ROW
   BEGIN
     SET NEW.date_modification = NOW();
   END$$
   DELIMITER ;
   ```

3. Vérifier le trigger en modifiant un client.

   ```
   UPDATE acheteurs SET adresse='Marrakech' WHERE id=2;
   ```

Partie 5 : Optimisation SQL

1. Créer la table commandes avec :
   - id INT AUTO_INCREMENT PRIMARY KEY
   - client_id INT
   - date_commande DATE

- montant DECIMAL(10,2)

```
CREATE TABLE commandes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  client_id INT,
  date_commande DATE,
  montant DECIMAL(10,2)
);
```

2. Insérer au moins **10 commandes** réparties sur les clients existants.

```
INSERT INTO commandes (client_id, date_commande, montant) VALUES
(1,'2024-01-01',500),(2,'2024-01-02',300),(1,'2024-01-03',200),
(2,'2024-01-04',150),(1,'2024-01-05',800),(2,'2024-01-06',450),
(1,'2024-01-07',120),(2,'2024-01-08',600),(1,'2024-01-09',700),
(2,'2024-01-10',250);
```

3. Écrire une requête pour **trouver toutes les commandes des clients d'Agadir** en utilisant **EXISTS** au lieu de IN.

```
SELECT * FROM commandes c
WHERE EXISTS (
  SELECT 1 FROM acheteurs a
  WHERE a.id = c.client_id
  AND LOWER(a.adresse)='agadir'
);
```

4. Créer un **index** sur la colonne client_id dans commandes.

```
CREATE INDEX idx_client_id ON commandes(client_id);
```

5. Écrire une requête avec **INNER JOIN** pour lister nom, prenom et date_commande pour toutes les commandes.

```
SELECT a.nom, a.prenom, c.date_commande
FROM acheteurs a
INNER JOIN commandes c ON a.id = c.client_id;
```

6. Utiliser **EXPLAIN** pour analyser la requête et indiquer si l'index est utilisé.

```
EXPLAIN
SELECT a.nom, a.prenom, c.date_commande
FROM acheteurs a
INNER JOIN commandes c ON a.id = c.client_id;
```

Partie 6 : Sécurité MySQL

Exercice 6 : Gestion des utilisateurs et privilèges

1. Créer un utilisateur ali sur localhost avec un mot de passe sécurisé.

```
CREATE USER 'ali'@'localhost' IDENTIFIED BY 'Ali@2024#Secure';
```

2. Accorder uniquement les privilèges SELECT et INSERT sur tp_eshop_db.acheteurs.

```
GRANT SELECT, INSERT ON tp_eshop_db.acheteurs TO 'ali'@'localhost';
```

3. Vérifier les privilèges de l'utilisateur.

```
SHOW GRANTS FOR 'ali'@'localhost';
```

4. Modifier le mot de passe de ali.

```
ALTER USER 'ali'@'localhost' IDENTIFIED BY 'Ali@2025#New';
```

5. Créer un rôle role_consultation avec le privilège SELECT sur tp_eshop_db.*.

CREATE ROLE role_consultation;

GRANT SELECT ON tp_eshop_db.* TO role_consultation;

6. Assigner le rôle à l'utilisateur ali.

   GRANT role_consultation TO 'ali'@'localhost';

Exercice 7 : Prévention injection SQL

1. Créer une requête préparée pour sélectionner un client par id.

   PREPARE stmt FROM 'SELECT * FROM acheteurs WHERE id = ?';

2. Exécuter la requête préparée avec id=2.

   SET @id = 2;

   EXECUTE stmt USING @id;

   DEALLOCATE PREPARE stmt;

Partie 7 : Questions de réflexion

1. Pourquoi ne faut-il pas utiliser SELECT * sur de grandes tables ?
   SELECT * consomme plus de ressources car il récupère toutes les colonnes, même celles inutiles, ce qui ralentit l'exécution sur de grandes tables.
2. Quelle est la différence entre une **procédure** et une **fonction** en MySQL ?
   Une procédure stockée exécute des traitements sans retourner de valeur directement, alors qu'une fonction retourne obligatoirement une valeur exploitable dans une requête.
3. Qu'est-ce qu'un **trigger** et dans quel cas serait-il utile ?
   Un trigger est un mécanisme automatique qui s'exécute lors d'un INSERT, UPDATE ou DELETE afin d'appliquer une règle sans intervention de l'utilisateur.
4. Pourquoi utiliser un **index** sur une colonne fréquemment utilisée dans WHERE ou JOIN ?
   Un index accélère l'accès aux données en réduisant le nombre de lignes parcourues lors des clauses WHERE et des jointures.
5. Explique la différence entre **PRIMARY KEY** et **UNIQUE**.
   Une PRIMARY KEY identifie de façon unique chaque ligne et interdit les valeurs NULL, tandis qu'une contrainte UNIQUE accepte une valeur NULL.

6. Que se passe-t-il si on oublie le WHERE dans un UPDATE ou DELETE ?
   Sans WHERE → toutes les lignes sont affectées.
   Oublier la clause WHERE dans un UPDATE ou DELETE entraîne la modification ou la suppression de toutes les lignes de la table.
7. Pourquoi les colonnes de type texte très long ne sont pas recommandées pour un index ?
   Texte long = index volumineux et lent.
   Les colonnes texte longues produisent des index volumineux qui ralentissent les opérations de recherche et augmentent l'espace disque utilisé.
8. Explique la différence entre **IN**, **OUT** et **INOUT** dans les procédures.
   Le paramètre IN reçoit une valeur, OUT renvoie une valeur, et INOUT permet à la fois l'entrée et la sortie de données.

9. Quelles sont les différences entre **INNER JOIN** et **LEFT JOIN** ?
   INNER JOIN retourne uniquement les lignes correspondantes dans les deux tables, tandis que LEFT JOIN retourne toutes les lignes de la table de gauche.
10. Pourquoi l'usage de fonctions comme UPPER() ou LOWER() dans le WHERE peut ralentir une requête ?
    L'utilisation de fonctions comme UPPER() ou LOWER() dans le WHERE empêche MySQL d'utiliser les index efficacement.
11. Quand utiliser EXISTS plutôt que IN ?
    EXISTS est plus performant que IN lorsque la sous-requête retourne un grand nombre de résultats.
12. Quelle est la différence entre UNION et UNION ALL ?
    UNION élimine les doublons dans le résultat, alors que UNION ALL conserve toutes les lignes sans vérification.
13. Pourquoi est-il important d'utiliser des **requêtes préparées** pour les données entrées par les utilisateurs ?
    Les requêtes préparées protègent contre l'injection SQL en séparant les données du code SQL.
14. Quelle est la différence entre **table temporaire** et table normale ?
    Une table temporaire est automatiquement supprimée à la fin de la session, contrairement à une table normale.
15. Pourquoi limiter les privilèges d'un utilisateur à ce qui est strictement nécessaire (principe du "least privilege") ?

Limiter les privilèges renforce la sécurité en réduisant l'impact d'une attaque ou d'une erreur humaine.

16. Pourquoi ne faut-il pas permettre à root de se connecter à distance ?
    Autoriser root à distance augmente fortement le risque de piratage du serveur de base de données.
17. Quelles sont les différences entre **cache** et **buffer** dans MySQL et comment cela impacte la performance ?
    Le cache stocke des résultats déjà calculés, tandis que le buffer conserve temporairement les données en mémoire pour accélérer l'accès
18. Explique la différence entre les moteurs **InnoDB** et **MyISAM**.
    InnoDB supporte les transactions et l'intégrité référentielle, contrairement à MyISAM qui est plus simple mais moins sécurisé.
19. Pourquoi utiliser EXPLAIN pour analyser une requête SQL ?
    EXPLAIN permet d'analyser le plan d'exécution d'une requête afin d'identifier les problèmes de performance.
20. Que signifie DETERMINISTIC dans la création d'une fonction MySQL ?
    Le mot-clé DETERMINISTIC indique qu'une fonction retourne toujours le même résultat pour les mêmes paramètres.
21. Qu'est-ce qu'une **vue** et quels sont ses avantages par rapport à une requête directe ?
    Une **vue** est une requête stockée qui simplifie l'accès aux données et améliore la sécurité.
22. Quels sont les avantages et les inconvénients de l'utilisation des **triggers** dans une base de données ?
    Les **triggers** automatisent certaines actions mais peuvent compliquer la maintenance et le débogage.
23. Pourquoi utiliser des **tables temporaires** plutôt que créer une table normale pour des calculs intermédiaires ?
    Les tables temporaires évitent d'encombrer la base avec des tables intermédiaires inutiles.

24. Pourquoi % au début d'un LIKE ralentit la requête ?
    Un LIKE commençant par % empêche l'utilisation des index et ralentit la recherche.
25. Explique comment un rôle peut simplifier la gestion des utilisateurs dans MySQL.
    Les rôles simplifient la gestion des droits en regroupant plusieurs privilèges sous une seule entité.

26. Que fait la clause READS SQL DATA dans une fonction MySQL ?
    READS SQL DATA indique que la fonction peut lire les données sans les modifier.
27. Quelle est la différence entre un **cache de requêtes** et un **index** ?
    Un index accélère l'accès aux données, tandis que le cache stocke des résultats déjà calculés.
28. Explique le concept de **N+1 problem** et comment l'éviter.
    Le problème N+1 correspond à l'exécution excessive de requêtes et peut être évité en utilisant des JOIN.
29. Pourquoi les sauvegardes doivent-elles être sécurisées et protégées ?
    Les sauvegardes doivent être protégées car elles contiennent des données sensibles exploitables en cas de fuite.
30. Quelles sont les bonnes pratiques pour sécuriser un fichier de configuration MySQL (my.cnf ou my.ini) ?
    Les fichiers de configuration MySQL doivent avoir des droits restreints et, si possible, être chiffrés.
31. Comment SSL protège-t-il les connexions MySQL ?
    SSL chiffre les échanges entre le client et le serveur MySQL pour empêcher l'interception des données.
32. Pourquoi les requêtes DISTINCT et ORDER BY peuvent ralentir une grande table ?
    DISTINCT et ORDER BY ralentissent les requêtes car ils nécessitent des opérations de tri coûteuses.
33. Quelle est la différence entre CREATE USER et GRANT ?
    CREATE USER crée un compte, tandis que GRANT attribue des privilèges à un utilisateur ou un rôle.
34. Pourquoi utiliser des **variables locales** dans une procédure ou fonction MySQL ?
    Les variables locales améliorent la lisibilité du code et évitent les conflits de valeurs.
35. Quelles sont les implications d'une table avec **beaucoup de colonnes modifiées fréquemment** sur la performance si un index est présent ?
    Une table souvent modifiée avec des index subit une baisse de performance à cause des mises à jour répétées des index.